

## Aula 3 - NGINX & Proxying

### Docupedia Export

Author:Cardoso Cristian (CtP/ETS)

Date:28-May-2026 18:23

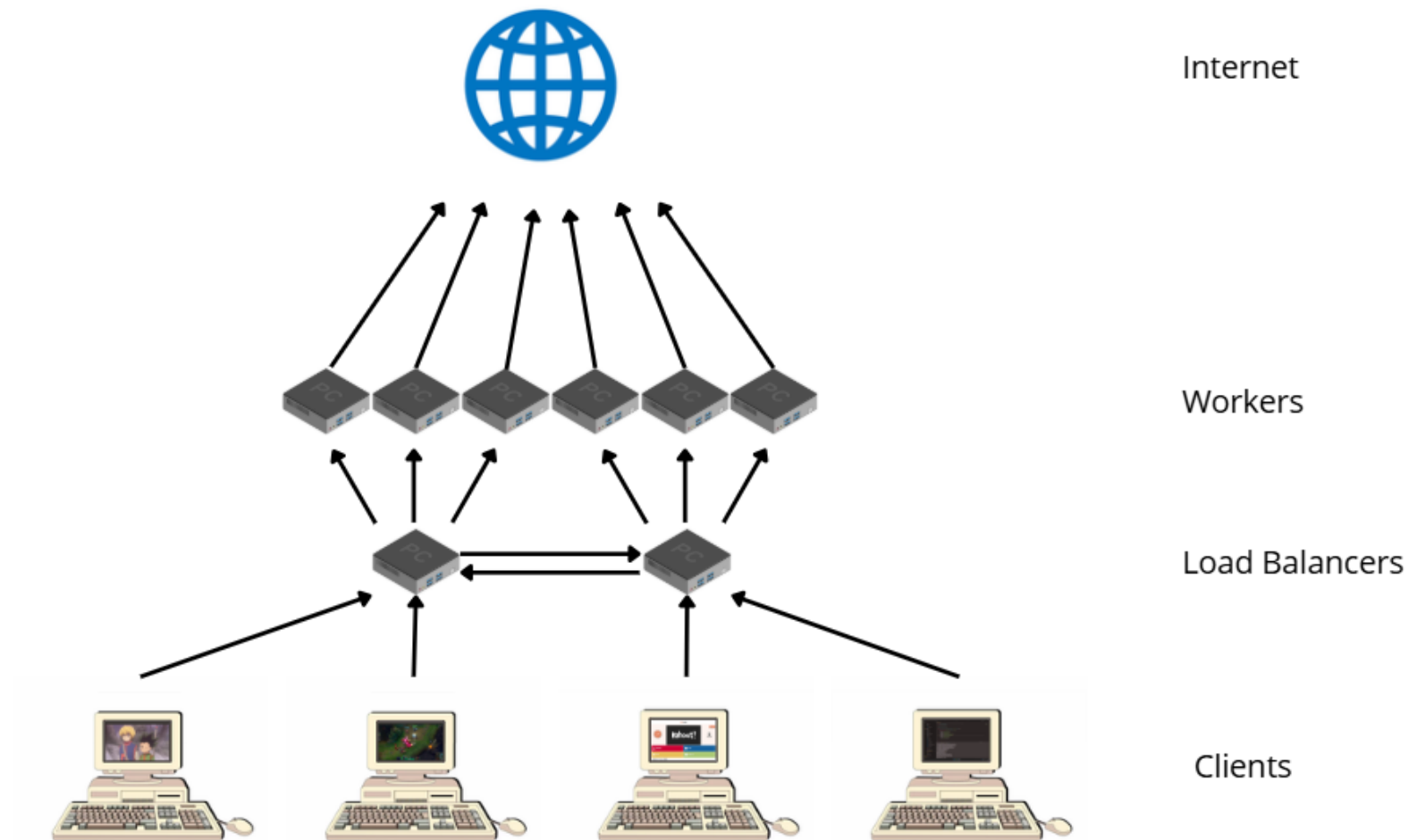
## Table of Contents

|                                               |           |
|-----------------------------------------------|-----------|
| <b>1 Forward Proxy</b>                        | <b>3</b>  |
| 1.1 Funcionamento de um Proxy de grande porte | 4         |
| 1.2 Clients                                   | 5         |
| 1.3 Load Balancers                            | 5         |
| 1.4 Workers                                   | 5         |
| 1.5 Internet                                  | 5         |
| <b>2 Reverse Proxy</b>                        | <b>6</b>  |
| <b>3 Prática</b>                              | <b>8</b>  |
| 3.1 Configuração do "nginx.conf"              | 8         |
| 3.2 Load Balancer                             | 9         |
| <b>4 Atividade (Parte 1)</b>                  | <b>10</b> |
| <b>5 Atividade (Parte 2)</b>                  | <b>11</b> |
| 5.1 Etapa 1                                   | 11        |
| 5.2 Etapa 2                                   | 11        |
| 5.3 Etapa 3                                   | 11        |

# 1 Forward Proxy

Primeiro vamos entender como funciona um proxy. Um proxy é um sistema intermediário que recebe uma requisição de um cliente e a encaminha para outro servidor, retornando a resposta para quem fez a solicitação. Dessa forma, o cliente não se comunica diretamente com o servidor final, mas sim através desse intermediário, podendo filtrar quais requisições, ou seja, sites podem ou não ser acessados

Esse tipo de arquitetura é frequentemente utilizado para centralizar o ponto de entrada das requisições, permitindo controlar, redirecionar ou distribuir o tráfego para diferentes serviços internos.



## 1.1 Funcionamento de um Proxy de grande porte

O **Client** tenta acessar a internet, porém ele está por trás de um proxy, portanto ele irá fazer uma requisição para o proxy informando qual site ele deseja acessar, e quem de fato vai acessar o site é o servidor proxy/worker e ele irá retornar a resposta do site para o Client.

O servidor proxy é um conjunto de computadores, possuindo Load Balancers para o gerenciamento das requisições e diversos computadores destinados ao processamento de requisições e retorno de respostas. Isso gera uma dúvida, toda vez que tento acessar um site estando por trás de um proxy é possível que minha requisição esteja sendo processada por diferentes computadores, uma vez que o Load Balancer envia para o melhor no momento, a resposta para isso é SIM, você pode observar isso com o IP de origem da sua requisição, [TESTE AQUI](#)

## 1.2 Clients

São os clientes, no contexto da rede de uma empresa que utiliza proxy, são os computadores de cada funcionário acessando a internet

## 1.3 Load Balancers

São os computadores responsáveis por gerir o fluxo de requisições feitas ao servidor proxy, clusterizando e direcionando cada requisição. Costuma-se ter ao menos dois, onde um monitora o outro para saber seu estado, desse modo caso um acabe falhando o outro rapidamente pega para si os workers e adiciona eles ao seu gerenciamento até que o outro **Load Balancer** retorne

## 1.4 Workers

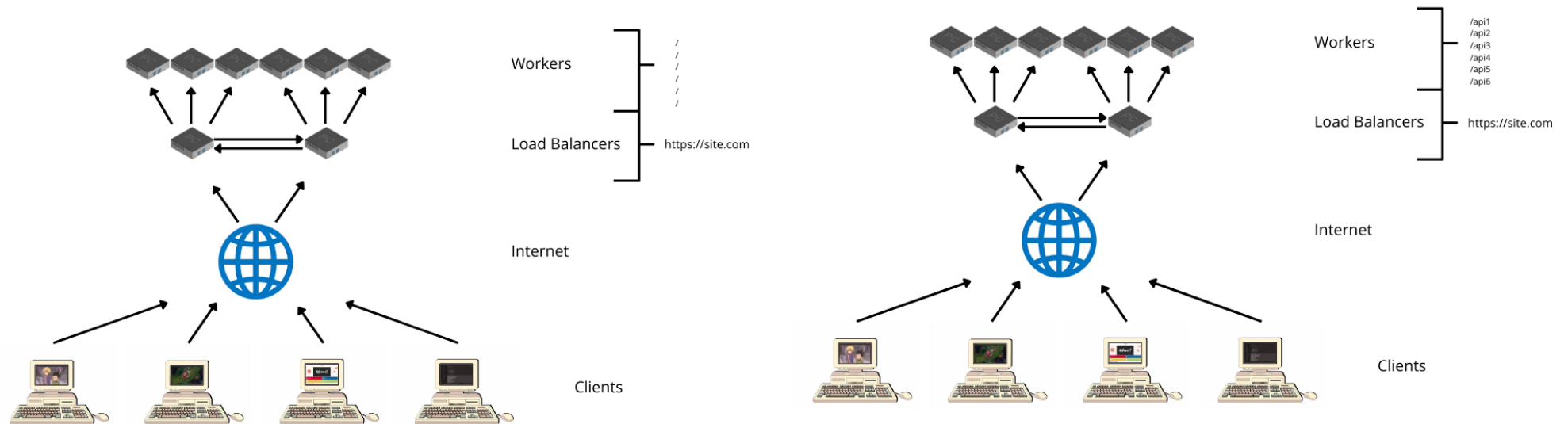
São computadores que trabalham juntos para dividir a carga gerada no servidor, deste modo temos vários workers e então não sobrecarregamos uma única máquina com todas as requisições

## 1.5 Internet

Rede de internet à qual todos os **Clients** vão de fato acessar, então caso haja alguma restrição ela vai ser aplicada a todos os **Clients**

## 2 Reverse Proxy

Um reverse proxy funciona de maneira inversa ao Forward proxy. Se Forward Proxy mascara o **Client** impedindo que ele seja descoberto pela internet, o **Reverse Proxy** mascara o servidor para que os **Clients** não o descubram. Um Reverse Proxy pode ser usado tanto para o gerenciamento de vários serviços através de um único domínio, quanto para evitar a sobrecarga de um único serviço utilizando instâncias desse serviço para distribuir as requisições em mais workers



## 3 Prática

Para a configuração do nosso Reverse Proxy vamos utilizar o **NGINX**

Vamos subir um compose que contém duas APIs e o "maestro" nginx, teremos duas instâncias da mesma "API", ou seja, dois workers que o nginx vai ter para distribuir as requisições recebidas. Vamos entender como funciona a configuração do **nginx**, ele depende o arquivo **nginx.conf**, portanto vamos começar por isso:

### 3.1 Configuração do "nginx.conf"

O primeiro bloco a ser definido na nossa configuração é o "events", que define o máximo de conexões totais nas nossas aplicações, ou seja, quantas pessoas podem acessar no total dos sistemas que estão sendo gerenciados

O **nginx** NÃO irá rodar sem este bloco

```
events {  
    worker_connections 1024;  
}
```

Agora iremos configurar o bloco "http", onde definiremos as aplicações que serão geridas pelo nosso "maestro", cada aplicação será um **upstream**, que nada mais é do que um grupo de servidores/containers que são acessados através de um único destino, é com isso que podemos fazer o que foi explicado a cima "evitar a sobrecarga".

```
http {  
    upstream <NOME_DO_SEU_SERVIÇO> {  
        server <HOST>:<PORT>;  
    }  
}
```

**\*\*IMPORTANTE** : O <HOST> é o host de onde o serviço está rodando, portanto dentro do contexto de docker, isso se refere ao container e a porta que o serviço esta exposto. Agora caso esteja usando o **NGINX** por conta dentro do seu computador normalmente, então ai sim será o HOST da máquina.

Agora que ja criamos as aplicações que irão rodar podemos definir de que modo elas vão fazer isso, ainda dentro do bloco "http", utilizaremos o bloco "server" com os seguintes campos:

listen - Indica em que porta o nosso servidor nginx irá rodar, ou seja, qual porta será nosso centro de distribuição para nossas aplicações.

location - é cada caminho possível a partir do nosso centro de distribuição, e qual upstream vai rodar a partir dele, acompanho de "http://" para indicar que é um protocolo de conexão

```
http {  
    server {  
        listen <PORT>;  
        location /<ROUTE> {  
            proxy_pass http://<NOME_DO_SEU_SERVICO>;  
        }  
    }  
}
```

```
}  
}  
}
```

## 3.2 Load Balancer

Onde Load Balancer se encaixa nisso tudo? Para entender isso precisamos entender um pouco melhor como é o funcionamento de um Load Balancer. Ambos os Load Balancers compartilham o mesmo IP virtual, aquele que o cliente enxerga, portanto o cliente não sabe que está acessando um Load Balancer, portanto, como é definido para qual deles a requisição do cliente será enviada? O navegador do cliente não sabe como mas sabe que vai acessar o servidor desejado, isso dependerá da configuração do roteador onde os Load Balancers estão, basicamente orquestrando como um Load Balancer de Load Balancers, isto é "**Alta Disponibilidade (HA)**" ou "**Failover Cluster**". No nosso caso, cada Load Balancer é um container nginx que opera simultaneamente, dividindo o peso das requisições entre si para dobrar a capacidade de tráfego, enquanto é garantindo que a carga será distribuída imediatamente caso um deles fique sobrecarregado ou saia do ar.

Um detalhe conceitual importante, embora tenha dito antes que eles se monitoram, expliquei assim apenas por ser mais prático, entretanto na realidade eles não se monitoram, é o nosso roteador o responsável por gerir se eles estão aptos a receberem requisições ou não, os Load Balancers de fato nem sabem a existência de outros como ele, ele apenas trabalha baseado no que recebe do roteador.

## 4 Atividade (Parte 1)

Aplicaremos isso dentro do nosso contexto de **Docker**, crie um compose com os no mínimo três serviços.

Crie seu docker-compose.yml e seu arquivo nginx.conf, estarão vazios a primeiro momento, os configuraremos em seguida

### docker-compose.yml

- maestro - utilize a imagem: **nginx:latest**. Caminho para o arquivo de configuração dentro do container: **/etc/nginx/nginx.conf:ro**
- worker0X - utilize apenas a seguinte configuração:

```
worker0X:  
  image: hashicorp/http-echo  
  command: "-text='MENSAGEM DO WOKRKER X'"
```

### nginx.conf

- Porta exposta por padrão do hashicorp: **5678**
- Utilizaremos ambos os containers criados como workers para o mesmo upstream, para fazer o controle de carga de uma única aplicação

Baseado na porta exposta e na location definida acesse [http://localhost:<PORTA\\_EXPOSTA\\_NO\\_COMPOSE><LOCATION>](http://localhost:<PORTA_EXPOSTA_NO_COMPOSE><LOCATION>) e observe o worker ser alterado

## 5 Atividade (Parte 2)

Agora que já entendemos como funciona vamos ver isso em maior escala. Dando continuidade a atividade anterior, agora teremos mais clientes acessando nosso proxy, iremos simular isso através da containerização, portanto iremos criar e configurar nossos **cliente**.

**Configuração inicial** - No seu docker-compose.yml adicione o seguinte serviço:

```
plankton:
  image: crcardd/plankton:latest
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
  ports:
    - 5555:5032
```

### 5.1 Etapa 1

Crie um projeto .NET

```
dotnet new console -n Client
```

Seu funcionamento deve ser o seguinte: a cada 1 segundo ele irá fazer uma requisição para o servidor proxy que configuramos, com o retorno (o worker o qual realizou a atividade para ele) ele deve imprimir no terminal

### 5.2 Etapa 2

Configure um Dockerfile para containerizar esse projeto

### 5.3 Etapa 3

Adicione 10 container que utilizam desse dockerfile. Observe os acessos através do container de monitoramento **plankton**